

Documento Proyecto 2

Integrantes:

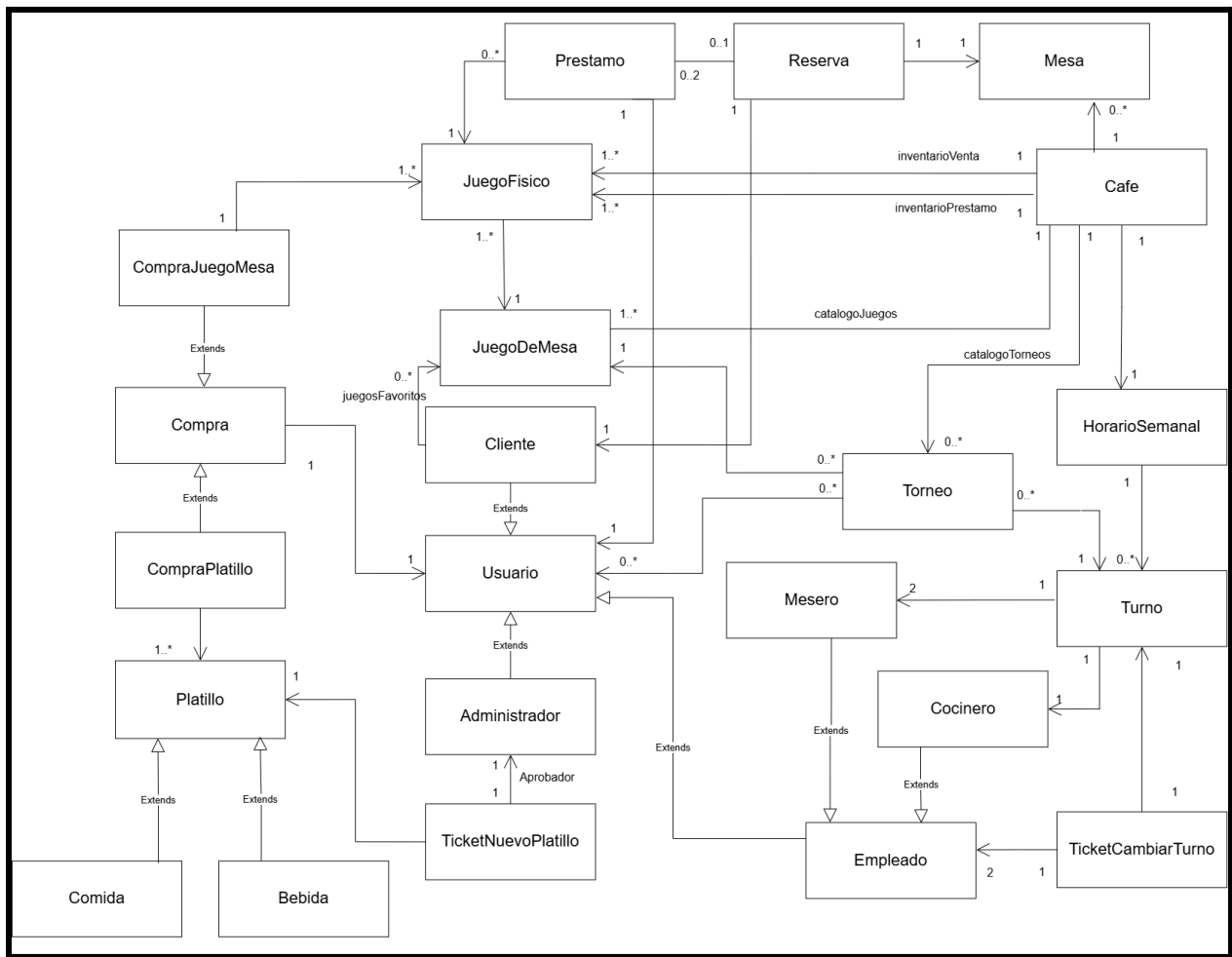
Carolina Ceballos Gómez - 202510267

Andrés Santiago Rodríguez Salazar - 202513997

Sebastián Felipe Angarita Rico - 202516784

Diagrama Diseño UML de alto nivel (nivel final):

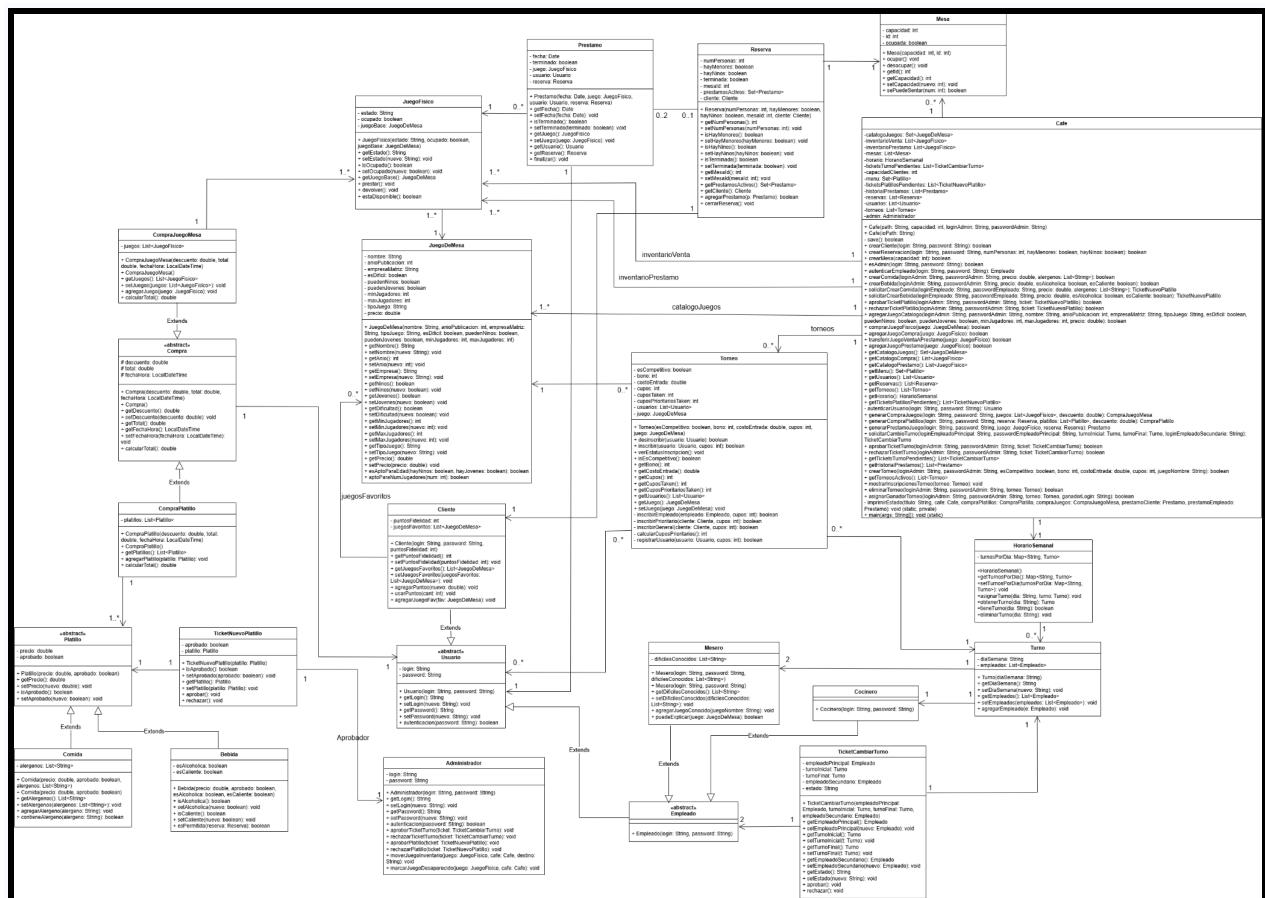
Objetos/Elementos, relaciones y cardinalidades



Este diagrama corresponde a la corrección y continuación del diagrama realizado en la entrega número 1. Para esta nueva versión, tomamos en cuenta todas las retroalimentaciones recibidas en la sustentación, en especial aquellas relacionadas con las cardinalidades que faltaban o no

Nota: En la carpeta de la Proyecto 2 se encuentra el archivo fuente .drawio, junto con otro .png para ver mejor el contenido de esta imagen.

Objetos/Elementos, atributos, métodos, relaciones y cardinalidades



Este diagrama evidencia la mejora y evolución del modelo que se presentó en la entrega 1. Para esta versión, se realizaron ajustes importantes basados en la retroalimentación recibida, especialmente en lo relacionado con la implementación de persistencia y la mejora en las cardinalidades. Se usó la interfaz Serializable en las clases para garantizar la persistencia del estado del sistema en los archivos, y Administrador pasó a ser una clase independiente. Gracias a las relaciones, cardinalidades, el hecho de que Cafe actúa correctamente como núcleo del sistema, hacen que el diagrama refleje los requerimientos funcionales del sistema.

Nota: En la carpeta de la Proyecto 2 se encuentra el archivo fuente .drawio, junto con otro .png para ver mejor el contenido de esta imagen.

Reglas del dominio (ajustadas al modelo)

- La cardinalidad de la relación Reserva -> Prestamo es (0..2). Es decir que una reserva puede separar máximo 2 juegos simultáneamente.
- Todas las reservas tienen un id de mesa válido.
- La suma de las personas de una nueva Reserva y el total de personas de las reservas activas no puede exceder la capacidadClientes del Café.
- Cuando se genere el subcatálogo para una reserva, todos los elementos en este deben tener la cantidad de personas en la reserva dentro de su rango.
- En el subcatálogo generado para una reserva, solo deben haber juegos que cumplan con las restricciones de niños y jóvenes.
- Un juego válido no puede tener el caso puedenNiños = true, puedenJovenes = false.
- Una reserva no puede tener hayMenores=false, hayNinos=true.
- Los juegos que estén en el subcatálogo aparecerán como disponibles si existe un juegoFisico en catalogoPrestamo con ocupado=false.
- No se puede crear un préstamo con un juegoFisico que este marcado con un ocupado=true.
- Un empleado solo puede realizar un préstamo si no está asignado al turno activo del día.
- Si el usuario de un préstamo es un cliente, es obligatorio que tenga una reserva asociada.
- Un préstamo solo se puede realizar si hay al menos 1 copia del JuegoFisico disponible (ocupado=false).
- Cada turno debe tener al menos 1 cocinero y 2 meseros asignados. Y por ende no se puede aprobar un cambio de turno si deja el café con menos de 1 cocinero o 2 meseros.
- Las bebidas alcohólicas no pueden ser vendidas a mesas donde hayan menores de edad (hayMenores=true).
- Las bebidas calientes no pueden ser vendidas a reservas con préstamos de juego tipo Acción.
- Los juegos tipo Acción no pueden ser prestados a reservas con bebidas calientes.
- El descuento de empleados es del 20% para ellos y 10% para los clientes cuando lo comparten.
- Todas las compras de juegos deben incluir un IVA del 19% en su valor total.
- Todas las compras de comida deben incluir un impuesto al consumo del 8% en su valor total.
- El administrador no puede alquilar juegos ni hacer compras.
- Solo el administrador puede mover juegos entre los inventarios.

- Cada compra le genera un 1% de su valor en puntos de fidelidad al cliente.
- Cada punto de fidelidad equivale a \$1.
- Los puntos de fidelidad solo pueden usarse como descuento en compras futuras.
- El estado de un juego (Nuevo, bueno, falta una pieza, etc) solo puede ser modificado por el administrador.
- Si no hay un mesero capacitado para un juego difícil, se puede prestar, pero con una advertencia.
- En las ventas, la propina sugerida es 10% antes de impuestos, pero el cliente puede cambiarla.
- Los premios de los torneos amistosos serán entregados en forma de puntos de fidelidad que los clientes podrán usar en compras, eso se puede hacer dado que los puntos corresponden a una cantidad monetaria fija, por ende, no se genera descuento a eso.
- Se entregarán de manera física los premios de los torneos competitivos y se compran en el momento, no se va a guardar un inventario, por consecuencia, el sistema no se encarga de darle los premios a los ganadores.

Requerimientos Funcionales

Estos requerimientos se encuentran explicados y detallados con las historias de usuario en un documento aparte (.../.../Proyecto-2/Entrega-Unica/Historias de Usuario - Proyecto 2).

REQS del usuario:

- **RF2:** Inicio de sesión
- **RF3:** Cerrar sesión
- **RF13:** Inscribir torneo
- **RF14:** Desinscribir torneo
- **RF15:** Consultar torneos

Cliente:

- **RF1:** Registro
- **RF4:** Crear reserva
- **RF5:** Ver catálogo de juegos
- **RF6:** Seleccionar juegos para préstamo
- **RF7:** Solicitar préstamo
- **RF8:** Devolver juegos
- **RF9:** Finalizar reserva
- **RF10:** Compra de juegos
- **RF11:** Compra de platillos
- **RF12:** Usar puntos de fidelidad
- **RF29:** Agregar a lista de favoritos

REQS del empleado:

- **RF16:** Consultar turno semanal
- **RF17:** Solicitar cambio de turno
- **RF18:** Sugerir nuevo platillo
- **RF19:** Compra con descuento de empleado
- **RF20:** Compartir descuento con clientes

REQS del mesero:

- **RF21:** Explicar un juego a clientes

REQS del administrador:

- **RF1:** Registrar Cliente
- **RF22:** Modificar catálogos
- **RF23:** Resolver sugerencias
- **RF24:** Resolver cambios de turno
- **RF25:** Mover juegos entre catálogos
- **RF26:** Reparar juegos
- **RF27:** Marcar juego como desaparecido
- **RF28:** Crear torneo
- **RF30:** Asignar ganador torneo
- **RF31:** Eliminar torneo
- **RF32:** Consultar inscripciones torneo

Asignación de Responsabilidades

Se explicarán las responsabilidades que cada clase debe cumplir y su rol u objetivo en la implementación.

Café: es la clase encargada de gestionar todos los catálogos del sistema, también mantiene el historial de préstamos y de compras. Además, debe asegurarse de validar la capacidad máxima del establecimiento antes de que se haga una reserva. También, por medio de esta se tiene acceso a mesas, juegos y platillos que se pueden ofrecer, es como el centro de control que coordina los diferentes subsistemas.

JuegoDeMesa: es la clase encargada de representar toda la información asociada a un juego, y es por medio de esta información que se definen las condiciones para su uso (ejemplo las reglas, el tipo, número de jugadores o edad mínima, etc). Su responsabilidad principal es definir estas restricciones para luego ser validadas al momento del préstamo.

JuegoFísico: esta clase se encarga de representar la copia física de un juego, no contiene su información (esta responsabilidad la tiene la clase JuegoDeMesa), pero sí puede indicar la disponibilidad y el estado de este. Las razones por las que se hizo esta separación serán explicadas y justificadas en la siguiente sección de este documento.

Usuario: es fundamental para el módulo de interacción con personas, pues sirve como la base de todos los roles del sistema (clase padre de cliente y empleado). También es el encargado de gestionar la autenticación del sistema, es decir, de cumplir con los requerimientos funcionales de login y contraseña.

Cliente: tiene 2 responsabilidades principales, gestionar y utilizar los puntos de fidelidad (que se irán acumulando con las compras) y hacer las reservas, compras, pagos, etc.

Empleado: su responsabilidad principal es consultar su turno de trabajo, revisando su horario y asegurándose de utilizar el ticket para cambio de turno en caso de ser necesario. Además de que puede realizar otras operaciones según su rol, es decir, si es mesero o cocinero, pues algunas responsabilidades varían.

Mesero: además de cumplir con las responsabilidades de la clase de empleado, esta clase está encargada de explicar los juegos difíciles (teniendo en cuenta la lista de juegos conocidos), también debe gestionar y validar otras operaciones en relación con préstamos, compras y pagos.

Cocinero: en comparación con la clase mesero, sus responsabilidades en el sistema son pocas, pero están concentradas en la operación interna de los platillos, de la disponibilidad y además de las propuestas (con el ticket de nuevo platillo).

Administrador: esta clase se encarga de gestionar todos los catálogos del sistema, también controla el inventario de juegos. Además, es quien aprueba o rechaza las solicitudes (como los cambios de turno o los nuevos platillos), es la interconexión de los demás roles de Empleado con el resto del sistema. Como responsabilidad general, debe supervisar el correcto funcionamiento de todo el sistema.

Reserva: las responsabilidades principales de esta clase giran en torno a la validación de todas las condiciones para hacer una reserva, por ejemplo, restricciones de edad, capacidad de mesa, etc. También debe registrar la ocupación de esta mesa para manejar la coherencia con la capacidad máxima del café. Y luego debe también asociar los préstamos y compras a la reserva. Después de realizar todas estas validaciones, debe determinar si la reserva puede finalizarse correctamente (juegos devueltos y compras pagadas, etc).

Prestamo: además de asociar los juegos con un usuario y una reserva, debe gestionar si el préstamo está activo o no lo está y también validar la disponibilidad de un juego antes de ser prestado, esto incluye también revisar las restricciones del juego según la reserva y todas las condiciones que se deban cumplir.

Mesa: debe representar el espacio físico que será ocupado en el café, pero principalmente se encarga de definir la capacidad máxima de personas permitidas, se complementa y se asocia con una reserva. Y es la encargada de hacer la conexión entre reserva y el café, que es el centro de control o coordinación del sistema.

Compra: sus responsabilidades se enfocan en los cálculos del valor total, de aplicar descuentos correspondientes (sean por los compartidos por empleados o por utilizar puntos de fidelidad), y además de esto debe registrar esta transacción en el sistema para que quede en el historial del cliente y pueda acumular más puntos de fidelidad.

CompraPlatillo: gestiona la compra de los productos del menú (de esta manera relaciona reservas con platillos o bebidas), además de que está encargada de revisar las restricciones de edad o de contexto (por ejemplo, juegos de mesa tipo acción).

CompraJuegoMesa: gestiona la compra de juegos físicos, pero primero debe revisar su disponibilidad en el catálogo de ventas y también aplicar los debidos impuestos.

HorarioSemanal: es la clase encargada de gestionar la distribución de los turnos de los empleados, al cumplir con sus responsabilidades garantiza que la clase de empleados también pueda cumplir las suyas.

Turno: es la representación de un turno de trabajo específico y es por esto que se debe asociar un empleado con un horario determinado.

TicketCambiarTurno: es crucial para cumplir con el registro de solicitudes de cambio, por medio de esta también se gestionan y revisan el estado de la solicitud y también se validan las condiciones necesarias para el cambio.

Platillo: representa un producto del menú y su responsabilidad consiste en definir el precio y, en caso de que sea un platillo nuevo, definir su estado.

Bebida: representa también un producto del menú (subclase de platillo), pero además de definir algunas características específicas como si son alcohólicas o la temperatura de estas. Esto anterior lo hace pues debe validar las restricciones que estas características conllevan.

Comida: representa también un producto del menú (subclase de platillo), pero además define la lista de alérgenos que puede contener, para hacer validaciones de acuerdo al cliente que realiza el pedido.

TicketNuevoPlatillo: se encarga de registrar todas las solicitudes de nuevos platillos y debe también gestionar el estado de aprobación de estos.

Torneo: representa y gestiona un torneo, incluyendo su información como el juego, la fecha, los cupos, también maneja la inscripción y desinscripción de participantes. Además tiene en cuenta los cupos para cada torneo, los que son para fanaticos, y luego también indica el ganador.

Justificaciones/Explicaciones del diseño

Se explicarán en detalle algunas de las decisiones tomadas al momento del diseño y la implementación, esto con el objetivo de que se pueda entender por completo el diseño y las implicaciones que estas decisiones puedan tener sobre él.

- 1) **Separación entre JuegoDeMesa y JuegoFísico:** se decidió hacer esta separación para definir la información conceptual de cada juego y no tener una repetición innecesaria de esta información (por esto se creó la clase JuegoDeMesa), ya que esta información no es tan fundamental al momento de hacer un préstamo, pues atributos como la empresa o el año en el que se publicó no tienen restricciones ni condiciones que se deban cumplir. Mientras que JuegoFísico es la manera en que evitamos la duplicación de datos y nos permite manejar más fácilmente las copias de un mismo juego pero en diferentes estados o versiones. Esta decisión nos ayuda a manejar el inventario sin complicaciones o repetición innecesaria y masiva de información conceptual (empresa, año de publicación, etc).
- 2) **Herencia en Usuario:** se decidió crear una jerarquía entre roles para distribuir mejor las responsabilidades, pues hay responsabilidades o funcionalidades que todos deben tener (como la autenticación con el login y la contraseña), pero luego de centralizar esto, se va bifurcando la implementación del control de acceso por roles. Esta decisión también nos ayuda a evitar duplicación de código y a definir mejor sus obligaciones.
- 3) **Separación en los tipos de compra:** se decidió separar estos tipos de compra (en CompraPlatillo y CompraJuegoMesa) pues cada uno de estos tiene reglas diferentes de negocios, por ejemplo, las restricciones o los impuestos que se deben utilizar al calcular el valor total. Esto evita que se utilicen condiciones erróneas en compras de tipos diferentes, y mejora la diferenciación entre estas clases.

- 4) **Centralización en la clase Café:** aunque se nos fue recomendado en la retroalimentación de la primera entrega dividir las tareas de esta clase en subclases (y si lo realizamos parcialmente), también consideramos que esta clase es el punto central de nuestro sistema, pues gestiona catálogos, reservas, préstamos y compras. Aunque esta acumulación de responsabilidades en una sola clase (definidas en la sección anterior del documento) nos complica un poco la implementación, también nos ayuda a reducir el acoplamiento entre clases y facilita la coordinación de operaciones complejas, actúa casi como un controlador del sistema.
- 5) **Separación de la clase Platillo en Bebida y Comida:** la decisión fue tomada principalmente para diferenciar las reglas específicas de cada clase (alcohol, temperatura, alérgenos). Pues de esta manera podemos hacer las validaciones necesarias sin tener que afectar el manejo de todos los productos del menú.
- 6) **Uso de tickets para solicitudes hacia el administrador:** consideramos que son fundamentales para nuestro diseño e implementación, pues de esta manera modelamos los procesos de aprobación, ya que al separar los estados la solicitud y la ejecución mejoramos la organización de nuestro sistema.
- 7) **Manejo de reglas en entidades:** decidimos poner todas las reglas del dominio como restricciones de edad, capacidad, disponibilidad, etc. dentro de las mismas entidades, para que la lógica no estuviera dispersa, sino que se hicieran las validaciones correspondientes en los lugares designados, de esta manera mejoramos la cohesión del sistema y también en el futuro podrá facilitarnos el mantenimiento del código en caso de cambiar alguna restricción o condición.

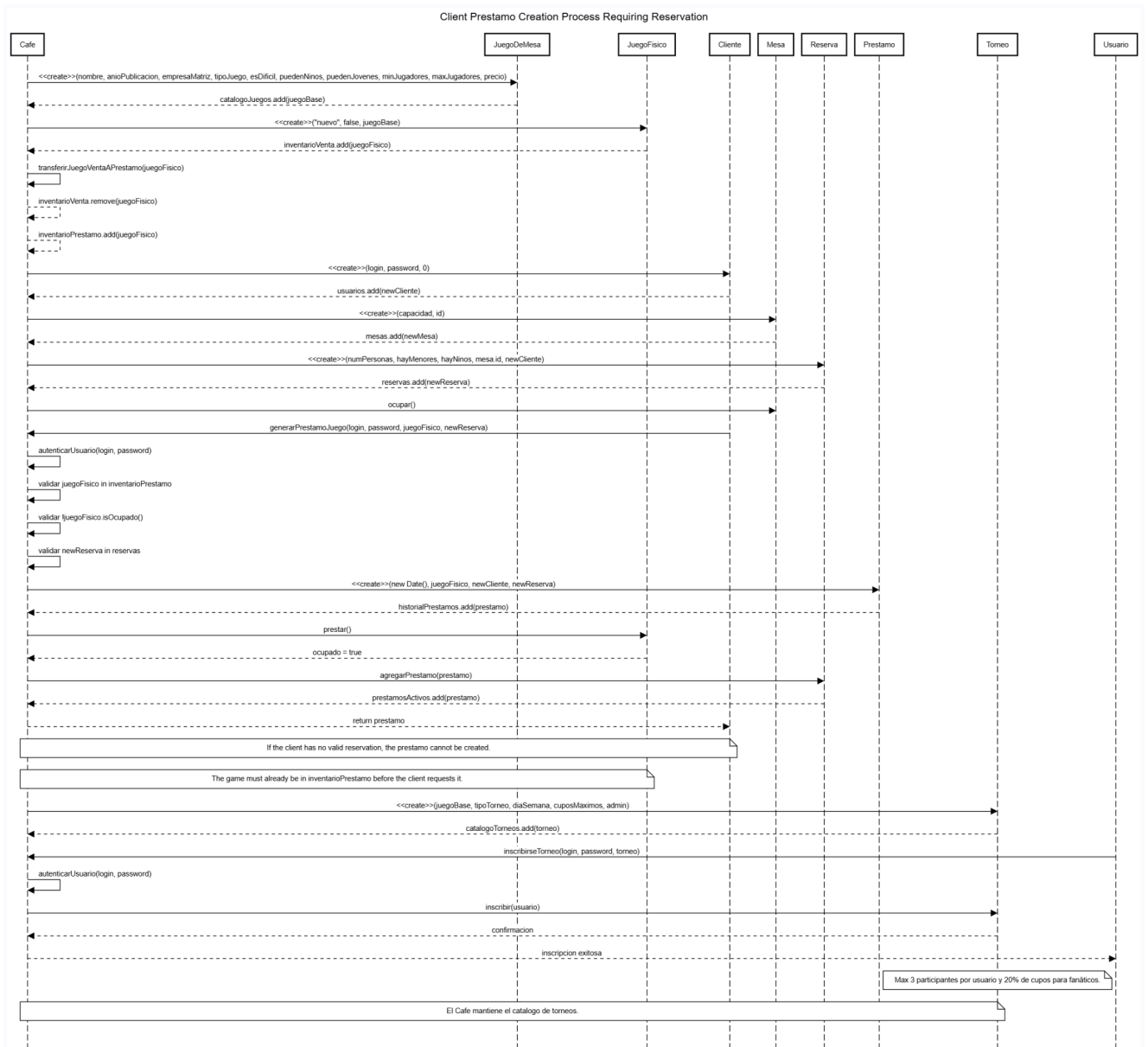
Restricciones del Proyecto

- **Lenguaje de programación:** El sistema debe ser implementado en Java debido a lo que nos pide el curso sobre diseño y programación orientada a objetos. Además, esto garantiza el uso de un solo lenguaje para mantener unificado el código y alinearse con los conocimientos de la clase.
- **Persistencia en archivos:** Toda la información como usuarios, juegos, reservas, préstamos, compras, etc, será persistida en archivos planos o JSON, para que cuando el programa se cierre y se vuelva a usar, toda la información debe estar correctamente. Al no hacer uso de bases de datos, se mantiene la simplicidad del proyecto y permite enfocarse en la lógica de la aplicación.
- **Consistencia del formato:** El formato de los archivos debe ser consistente en toda la aplicación. Esto significa que si se elige JSON, toda la información debe persistir siguiendo una estructura uniforme para facilitar su lectura y escritura.
- **Interfaz de usuario:** La aplicación no es necesario que tenga interfaz gráfica, el usuario

podrá interactuar con el programa a través de la consola, haciendo pruebas que demuestren el completo funcionamiento de la aplicación.

- **Tiempo de desarrollo:** El proyecto debe desarrollarse durante unas semanas asignadas, con dos entregas, análisis y entrega. Haciendo que el tiempo sea limitado y se necesite planificación del trabajo para cumplir con el proyecto.
- **Tamaño del equipo:** El grupo de trabajo para realizar el proyecto es de 3 personas, definiendo una dinámica de trabajo en donde habrá distribución de tareas y trabajo en equipo.
- **Control del proyecto:** Todo el código, documentos, archivos y versiones del proyecto se almacenarán en Git en un repositorio GitHub, permitiendo el trabajo colaborativo y el seguimiento de cambios.
- **Sin programas externos:** El programa no usará sistemas de pago externos, APIs u otros sistemas. Todos los trámites y procesos son simulados dentro del sistema y no requieren procedimientos con el mundo real.
- **Sin correo electrónico:** No se requerirán correos electrónicos a los usuarios para registrarse, confirmar compras o notificaciones. Toda la comunicación con el usuario se realizará por la interfaz de consola.
- **Multiplataforma:** La aplicación debe poder ejecutarse en cualquier sistema operativo que tenga instalada una Máquina Virtual de Java (JVM), esto se logra al mantener todo el programa en Java.
- **Control de acceso por rol:** El sistema controlará el acceso a funcionalidades según el rol del usuario (cliente, empleado, administrador), ya que un cliente no puede acceder a funciones de un administrador y al revés. Cada rol tendrá acceso a sus requerimientos funcionales específicos.
- **Privacidad de los datos:** Los datos, como información de login, compras, juegos favoritos, etc, de los clientes no serán visibles para otros clientes. Solo el administrador tendrá acceso a esto.

Diagrama de secuencia o escenarios de interacción



Este diagrama de secuencia modela el proceso de cómo se crea un préstamo de un juego de mesa por parte de un cliente, además, consideramos como condición crítica la existencia de una reserva válida. Este escenario está asociado a algunos requerimientos del sistema (RF5, RF6 y RF7), ya que integra varias clases como Café, Cliente, Reserva, JuegoFisico y Prestamo, y muestra cómo estas pueden colaborar para cumplir las reglas del negocio.

En el flujo se ve cómo la clase Café coordina las operaciones principales (pues es nuestro centro de control), incluyendo la autenticación del usuario, la validación de disponibilidad del juego y la verificación de la reserva activa. Esto para corroborar que sí se están cumpliendo todas las condiciones necesarias.

El diagrama también muestra la creación del objeto Préstamo y la actualización del estado del juego cuando todas las condiciones se cumplen, para así considerarse los casos en los que el préstamo no se pueda hacer.

Además, el diagrama Torneo se maneja a través de Cafe, que es quien controla todo. Primero, el administrador crea el torneo y se guarda en el catálogo del sistema. Luego, cuando un usuario quiere inscribirse, lo hace por medio de Cafe, que valida sus datos y le pide a Torneo que haga la inscripción. Así, Torneo se encarga de manejar los cupos, mientras Cafe sigue siendo el que coordina todo el proceso.

Justificación de Persistencia

La persistencia de nuestro sistema la diseñamos utilizando la serialización de objetos de Java, implementando la interfaz Serializable en la clase Cafe. Esta decisión la tomamos pues porque nos permite almacenar el estado completo del sistema en un archivo y recuperarlo después sin necesidad de reconstruir los componentes. Como Cafe actúa como nuestro centro de control y contiene todos los métodos de reservas, mesas, etc, al serializar esta clase ya logramos persistir toda la estructura del sistema. Esto simplifica mucho la persistencia.

Con el atributo ioPath definimos la ruta del archivo donde se va a almacenar el estado del objeto. Esto permite que la ubicación del archivo se pueda modificar. Además, implementamos dos constructores: uno para inicializar el sistema desde cero y otro para cargar el estado previamente guardado. Este segundo constructor utiliza flujos de entrada (ObjectInputStream y FileInputStream) para deserializar el objeto almacenado, y así poder restaurar todos los atributos del sistema.

El método save() tiene la lógica de escritura del objeto, utilizando un ObjectOutputStream para serializar la instancia actual de Cafe en el archivo definido por ioPath. Este método retorna un valor booleano que indica si la operación fue exitosa, para permitir que otras partes del sistema puedan reaccionar a los fallos de persistencia.

Justificación Pruebas Unitarias

En el desarrollo de las pruebas unitarias decidimos enfocarnos en verificar que se estuvieran cumpliendo las reglas de negocio que definimos desde el diseño. Por eso organizamos las pruebas por módulos (cafetería, compras, horarios, juegos, reservaciones y usuarios), siguiendo así la misma estructura del proyecto, para mantener coherencia entre el diseño y la validación.

En el caso del módulo de cafetería, quisimos comprobar que la jerarquía de clases estuviera bien construida. Por ejemplo, probamos Platillo a través de sus subclases (Comida y Bebida) para

asegurarnos de que la herencia realmente estuviera funcionando como esperábamos. Además, probamos las restricciones de bebidas alcohólicas cuando hay menores en una reserva. Para el módulo de compras, decidimos centrarnos en los cálculos. Probamos diferentes escenarios, incluyendo casos donde no hay productos. También validamos que no se pudieran agregar elementos nulos.

En horarios, probamos la gestión de turnos como una estructura dinámica. Queríamos ver que fuera posible asignar, consultar y eliminar turnos sin problemas, pero también que el sistema se diera cuenta de inconsistencias, como días o turnos nulos. En el módulo de juegos probamos que en los torneos se pudieran validar cosas como la asignación de cupos y la prioridad de ciertos usuarios. También probamos préstamos asegurándonos de que no se pudieran crear en estados inválidos, como cuando no hay una reserva asociada. En reservaciones, nos enfocamos en el límite de préstamos activos o el comportamiento al cerrar una reserva. En usuarios probamos principalmente la lógica de autenticación y los comportamientos específicos de cada tipo de usuario. Por ejemplo, en los clientes validamos el manejo de puntos de fidelidad y en los meseros probamos la lógica de qué juegos pueden explicar.

Finalmente, en el módulo de control, que es nuestro centro de manejo de las demás clases, nos enfocamos en la lógica de autenticación, registro y coordinación entre módulos. Decidimos probar la autenticación de manera separada (administrador, empleados y clientes) para garantizar que cada tipo de usuario solo pueda acceder a las funcionalidades que le corresponden. También verificamos flujos completos como la creación de reservas y la gestión de tickets de platillos, ya que ahí es donde interactúan varias partes del sistema al mismo tiempo. Incluimos una prueba de persistencia para confirmar que el estado importante del sistema realmente se guarda y se puede recuperar correctamente.

Justificación Pruebas de Integración

Para estas pruebas de integración nos enfocamos en cubrir cada uno de los requerimientos funcionales y verificar que estuvieran funcionando correctamente. Las separamos en 6 módulos importantes y explicaremos cada uno de ellos a continuación:

Para las pruebas de las funcionalidades del administrador (RF1, RF22, RF23, RF24, RF25, RF26, RF27, RF28, RF30, RF31, RF32) construimos escenarios que validarán funciones conjuntas, por ejemplo empezamos verificando la autenticación, y que solo aquellos con el rol de admin puedan tener los accesos y permisos específicos. También el administrador está asociado con los Tickets (tanto de platillos como de cambio de turno), así que los escenarios cubren la gestión de estos cambios, pero también de los catálogos y de la nueva funcionalidad de torneos. Estas pruebas son cruciales pues cubren la lógica central de control y además las acciones como aprobar un platillo deben demostrar que efectivamente se agrega al menú o que mover un juego entre catálogos actualiza correctamente el inventario,

Para las pruebas de las funcionalidades de los clientes (RF1, RF4, RF5, RF6, RF7, RF8, RF9, RF10, RF11, RF12, RF29) decidimos tomar como caso base para cada uno un café con un archivo temporal, al que le agregamos cierto contexto a través de métodos como *crearReservaActiva*, esto prueba la integración con el paquete de control y los demás, como compras, reservas y préstamos. Por ejemplo, en el RF10 comprobamos que el inventario se actualice después de una compra. Usamos el atajo de crear un usuario básico, después generamos un juego y una compra correspondiente con otros métodos atajo. Después hacemos las comprobaciones apropiadas de que se haya actualizado, como que el juego se haya eliminado del inventario y que la compra se haya efectuado con el valor total.

Para las pruebas de las funcionalidades del Empleado (RF16, RF17, RF18, RF19, RF20), nos enfocamos en validar las interacciones con otros módulos como por ejemplo el de horario, compras y los tickets. Por ejemplo, para el RF16 verificamos que el empleado pueda consultar sus turnos asignados, pero que también que el empleado previamente registrado. Para los RF 17 y 18 comprobamos que los tickets estén funcionando como deberían, o sea, no solo importa que el ticket se cree, sino que quede en el estado correcto y en la lista adecuada para ser procesado después. En RF19 garantizamos más la precisión de aplicar el descuento a la compra y que se refleje en el monto a pagar.

Para el requerimiento relacionado exclusivamente con los meseros (RF21) que es cómo se agrega que un mesero sabe un juego difícil (diferente a la reserva de juegos difíciles), decidimos lo siguiente: tenemos 2 tests; uno se encarga de comprobar que, si se agrega correctamente, la comprobación funciona; el otro, de que retorne falso en caso de que no. Ambos tienen la misma estructura: creamos un café, un empleado y un juego. Después, comprobamos que un mesero no “pueda ayudar” a menos que se agregue con el método empleado::agregarJuegoConocido. Esto lo hacemos en cada uno de los tests, con un `assertTrue` y `assertFalse` respectivamente.

Para las pruebas de las funcionalidades del Usuario (RF2, RF3, RF13, RF14, RF15) nos enfocamos en validar el flujo y la conexión básica de los módulos, pues todas las interacciones deben funcionar conectando al usuario con las acciones. Primero verificamos tanto el caso exitoso como el fallido de inicio de sesión, no nos importa solo que nos retorne true o false, sino que el estado del sistema cambie correctamente, y que se cambie a un usuario válido y activo. Para el RF3, que probamos el cierre de sesión, lo fundamental aquí es que los datos se guarden en el usuario correspondiente. Luego, en RF13 y RF14 nos enfocamos en la inscripción y desinscripción a torneos. Aquí se combinan las restricciones de límite de participantes por usuario, disponibilidad de cupos y actualización del estado del torneo. Las pruebas nos ayudan a verificar que todas esas condiciones se respeten al mismo tiempo. Y finalmente para RF15, validamos que el sistema pueda devolver la información básica de los torneos disponibles.

Para las pruebas de las funcionalidades de la persistencia se construyó un escenario que cubre la integración de varios componentes del sistema, validando que el estado del café pueda guardarse en un archivo y recuperarse correctamente. Esta prueba verifica que después de crear un café con usuarios (clientes y empleados), reservas, juegos en catálogo, inventarios y un torneo activo, el método `save()` guarde toda esa información. Luego se mira que un nuevo objeto en `Cafe`, cargado desde el mismo archivo, tenga todos los datos como la autenticación del administrador, empleados registrados, la cantidad de usuarios y reservas, los catálogos y torneos. Gracias a esto, se asegura de que la persistencia guarda correctamente las relaciones entre el sistema, garantizando que se puedan guardar los estados completos después de cerrar el programa y al ejecutarse.

Justificación interfaces basadas en consola

La aplicación se organizó con una interfaz basada en consola, en donde la consola principal es un método `main` dentro de la clase `Cafe`. La responsabilidad del `main` es cargar un archivo de un café específico al que se le realizarán los cambios y mostrar un menú de bienvenida con las 3 opciones de ingreso: cliente, empleado y administrador. Una vez que el usuario seleccione una opción, la consola instancia la clase de las otras consolas correspondientes: `ConsolaCliente`, `ConsolaEmpleado`, `ConsolaAdministrador` y ejecutarlas. Esta separación permite que la lógica de autenticación de cuáles métodos puede usar cada usuario se encuentre al principio y no en su propia clase, siguiendo el principio de responsabilidad única y se facilite la lectura y posibles modificaciones en el código.

Cada consola específica se diseñó como una clase independiente que recibe una referencia al objeto `Cafe` en su constructor, lo que permite acceder a toda la lógica del sistema mediante los métodos públicos creados. Dentro de cada consola, se implementa una autenticación y se muestra un menú con las opciones disponibles según el rol del usuario. Esto garantiza que la interfaz de usuario esté separada de la lógica y la persistencia, las consolas solo se encargan de interactuar con el usuario a base de mostrar mensajes y leer entradas, mientras que la clase `Cafe` contiene los métodos y se encarga del manejo de datos. De esta forma, si en el futuro se quisiera reemplazar la interfaz de consola por una interfaz gráfica, solo sería necesario modificar las clases de consola sin afectar al sistema.

Nota: las instrucciones para la ejecución de la consola están en un documento aparte (.../.../Proyecto-2/Entrega-Unica/Instrucciones para ejecución proyecto en Eclipse).